Agentes: equipos multiagente y frameworks

Máster AI Engineer · Sesión 14 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico
02 Bloque práctico
03 Ejercicio para casa
04 Cierre y dudas



thePower
Tech School

01

Bloque teórico

thePower
Tech School

Del agente único al equipo de agentes

Un solo agente vale para tareas simples, pero **se satura cuando la tarea tiene varias fases**.

- **Uno que lo hace todo se vuelve un lío**: pierde el hilo y mezcla instrucciones
- **Mejor varios especializados**: cada uno con su rol, sus herramientas y su prompt
- **Uno investiga, otro escribe, otro revisa**, y el resultado sube de nivel
- **Ejemplo**: un informe de mercado con un agente que busca fuentes, otro que redacta y otro que verifica datos



ANALOGÍA

En vez de un empleado que hace de todo regular, **un equipo pequeño donde cada uno es bueno en lo suyo**: sabes a quién pedir cada cosa y cada pieza sale mejor.

thePower Tech School

Patrones de coordinación: el supervisor

SUPERVISOR
recibe la tarea y delega



Agente A
investiga



Agente B
redacta



Agente C
revisa

El supervisor decide **a quién pasar cada parte, en qué orden y cuándo parar**. Es el patrón más común porque es flexible y fácil de depurar.

thePower
Tech School

Otros patrones: jerárquico y secuencial



JERÁRQUICO

- **Equipos de equipos**, con varios niveles de mando
- **Un supervisor de supervisores** que reparte por áreas
- Para **problemas grandes** que se dividen en subproblemas
- **Ejemplo**: un área de investigación y otra de producción editorial



SECUENCIAL

- **Uno detrás de otro**, en cadena de montaje
- **La salida de uno entra directa** en el siguiente
- **Simple, predecible** y muy fácil de depurar
- **Ejemplo**: transcribir, resumir y traducir un acta

thePower Tech School

Memoria compartida y handoff

Los agentes tienen que **colaborar sobre la misma información**, no ir cada uno por su lado.

- **Handoff**: pasar el trabajo, y el contexto necesario, de un agente a otro
- **Estado compartido**: un único sitio donde todos leen y escriben
- Así **no se repiten, no se contradicen** y no pierden lo ya hecho
- **Ejemplo**: el investigador guarda las fuentes en el estado y el revisor las comprueba una a una



LA CLAVE

El **handoff**, ese paso limpio de una tarea de un agente a otro, es lo que **separa un equipo que funciona de un grupo de agentes descoordinados**.

thePower Tech School

Los tres frameworks: cada uno con su filosofía



LANGGRAPH
Control total con grafos
Ya lo conoces de la sesión 5
Máxima flexibilidad, más código
Ideal para flujos con bucles



CREWAI
Equipos por roles
Rapidísimo de montar
Menos control fino
Ideal para prototipos



AUTOGEN
Agentes que conversan
Diálogo entre agentes
De Microsoft
Ideal para debate y revisión

Mismo objetivo, tres formas de llegar: **grafo, roles o conversación**. No hay uno mejor; hay uno mejor para cada caso.

thePower Tech School

LangGraph vs CrewAI: control vs rapidez



LANGGRAPH

- Defines el **grafo paso a paso**: nodos y aristas
- **Control total del flujo**, bucles y condiciones
- Más código, **más potencia**
- **Elige esto si** el flujo es complejo o hay reintentos



CREWAI

- Defines **roles y tareas**, casi como un organigrama
- **El framework coordina solo** por ti
- Menos código, **menos control**
- **Elige esto si** quieres un prototipo hoy mismo

thePower Tech School

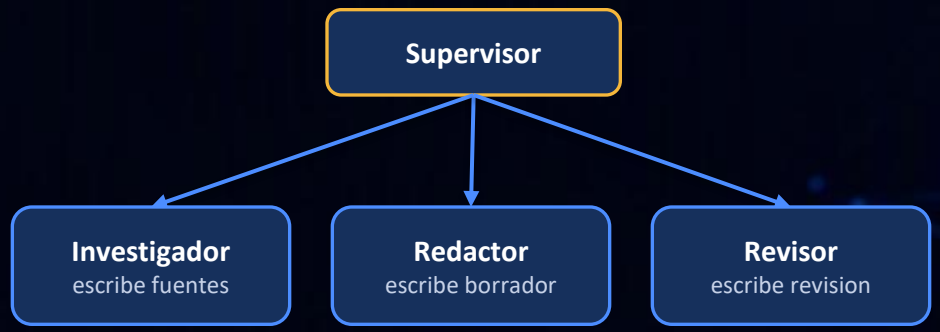
LangGraph vs CrewAI: un plano frente a un organigrama

MISMO EQUIPO
investigador, redactor y revisor

MISMA TAREA
resumir A2A con citas

MISMO MODELO
llama3.1 en local

LANGGRAPH · tú dibujas el plano



CREWAI · tú escribes el organigrama



Lo defines con	nodos, aristas y un estado	roles, objetivos y tareas
Quién decide el orden	tú: aristas y reglas del supervisor	el framework: secuencial o un manager
Handoff	una arista: todos vuelven al supervisor	context=[...]: una tarea alimenta a la siguiente
Memoria compartida	el Estado: fuentes, borrador, revision	la salida de cada tarea
Medido en clase	**110 líneas · 9 llamadas · 274 s · 7 citas**	**61 líneas · 12 llamadas · 815 s · 3 citas**

IDEA CLAVE En LangGraph dibujas el camino. En CrewAI describes el equipo y el framework decide el camino.

thePower Tech School

Los roles en acción: quién pasa el testigo a quién

LANGGRAPH · el testigo vuelve siempre al supervisor, que lee el estado y decide



Ejecución real: 5 turnos del supervisor y 9 llamadas al LLM. Las barandillas (tope de turnos, una reescritura como mucho) son código tuyo.

CREWAI (SECUENCIAL) · el testigo pasa de mano en mano, sin jefe: cada tarea recibe las anteriores



IDEA CLAVE En LangGraph el testigo vuelve al jefe. En CrewAI pasa de mano en mano.

thePower Tech School

Jerarquías: quién manda en LangGraph y en CrewAI

LANGGRAPH · el jefe lo escribes tú

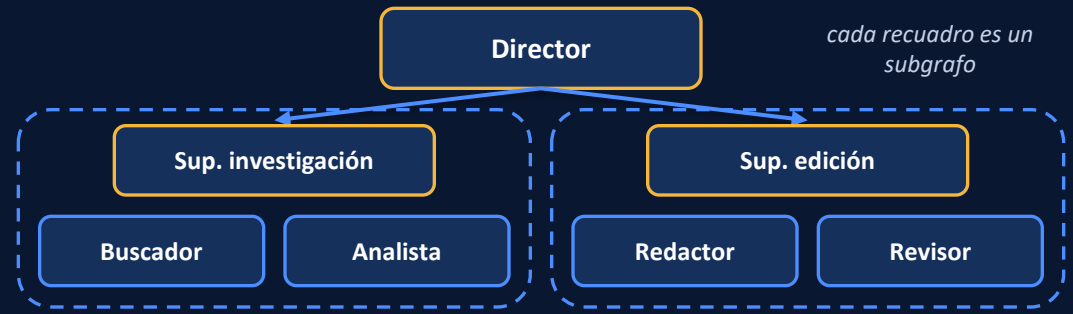
LO QUE MONTAMOS HOY · SUPERVISOR

tu código manda



JERÁRQUICO · SUPERVISOR DE SUPERVISORES

cada recuadro es un subgrafo



CREWAI · el jefe lo pone el framework

LO QUE MONTAMOS HOY · Process.sequential

Sin jefe: el orden es el de la lista de tareas



JERÁRQUICO · Process.hierarchical + manager_llm



IDEA CLAVE Más niveles = más control del jefe… y más llamadas al LLM.

thePower Tech School

A2A: que agentes de distintos sistemas se entiendan

Protocolo **Agent-to-Agent** (Google, 2025): un idioma común entre agentes.

- Un estándar para que los agentes **se descubran** entre sistemas
- **Anuncian qué saben hacer** con una ficha de capacidades
- **Se delegan tareas** sin conocerse de antemano
- **Ejemplo**: tu agente de viajes contrata al agente de una aerolínea



POR QUÉ IMPORTA

Como el **USB de los agentes**: un enchufe común para que agentes de cualquier empresa **colaboren sin integraciones a medida**.

thePower Tech School

Coordinar cuesta: errores y coste

Más agentes = más potencia, pero también más cosas que vigilar.

- **Manejo de errores**: si un agente falla a mitad, ¿qué pasa con el resto?
- **Coste**: cada agente consume tokens; cinco agentes cuestan como cinco
- **Latencia**: cada paso añade segundos de espera
- Usa **Ollama en local** o el **free tier de Gemini** para practicar sin gastar



IDEA CLAVE

Un equipo de agentes es más potente, pero **solo compensa cuando la tarea es de verdad compleja**. Para lo simple, un agente basta y sale más barato.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Qué hacemos hoy: una tarea, tres formas

1

AGENTE ÚNICO

el patrón de la sesión 5 con una herramienta de búsqueda

`paso 1 · 01_agente_unico.py`

2

LANGGRAPH

supervisor + investigador + redactor + revisor

`paso 2 · 02_equipo_langgraph.py`

3

CREWAI

el mismo equipo, definido por roles y tareas

`paso 3 · 03_equipo_crewai.py`

4

COMPARAR

una tabla con lo medido de verdad

`paso 4 · 04_comparar.py`

Mismo modelo (llama3.1 en local), misma búsqueda (DuckDuckGo) y misma forma de puntuar. **Si no, la comparación no vale.**

Spoiler honesto: el primer equipo no funcionó. El supervisor se quedó en bucle. Lo veremos.

thePower Tech School

La tarea: investigar A2A y escribirlo con formato

TAREA

Investiga qué es el protocolo A2A (Agent2Agent) y escribe un resumen estructurado para un equipo técnico: qué es, quién lo impulsa, cómo funciona y en qué se diferencia de MCP.

FORMATO EXIGIDO

`## Qué es · ## Quién lo impulsa · ## Cómo funciona · ## A2A vs MCP`

2-3 frases por título, cada dato con su fuente [n]

1. **Varias fases**
   buscar, filtrar, ordenar y redactar
2. **Medida con código**
   evaluar_informe(): secciones y citas
3. **Detector de mentiras**
   A2A: Google, abril 2025, hoy en la Linux Foundation

MCP = Model Context Protocol (Sesión 15)

Todo lo común vive en comun.py: la tarea, el formato, buscar_web() (la de la sesión 12) y evaluar_informe(). Los tres programas lo importan igual.

thePower Tech School

Paso 1 · Objetivo

1

OBJETIVO DEL PASO

Ver hasta dónde llega un agente solo ante una tarea de varias fases

DE LA TEORÍA

Slide 4: un solo agente se satura cuando la tarea tiene varias fases.

LO QUE VAMOS A VER

- El agente de la S5 con otra herramienta
- Cuántas búsquedas hace
- Qué tal le sale el informe

ARCHIVO

- 01_agente_unico.py

thePower Tech School

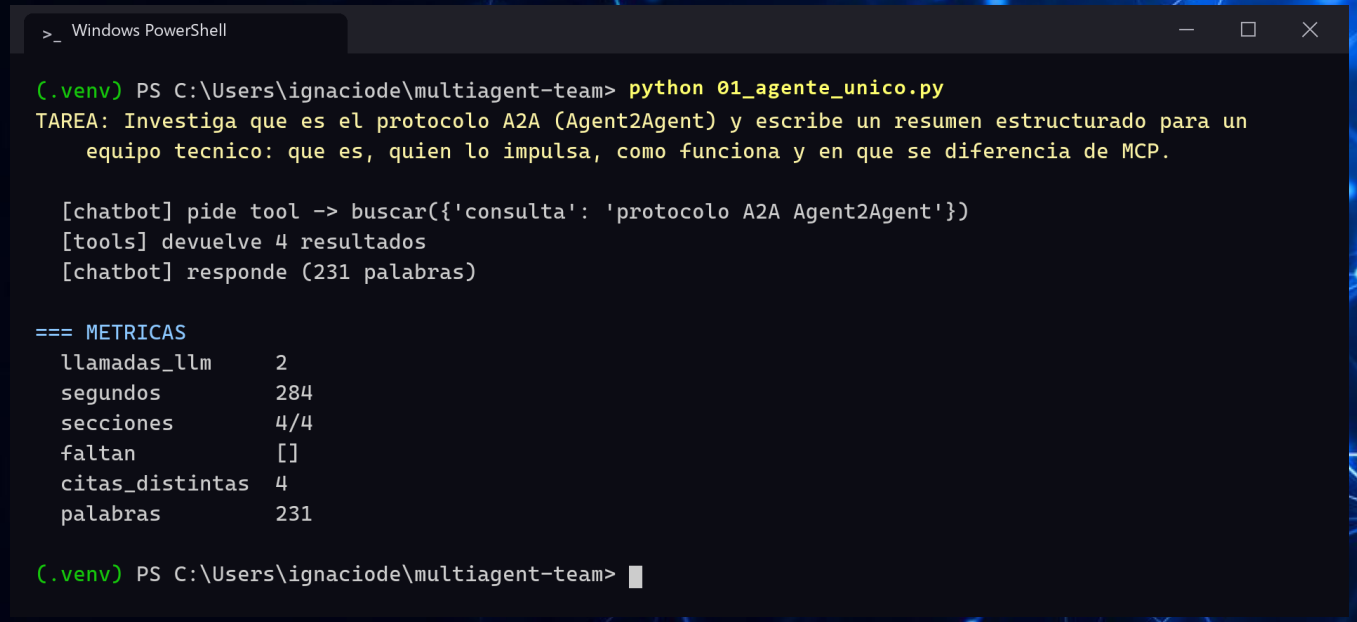
Paso 1 · Punto de partida: un agente único

QUÉ HACEMOS

- El grafo de la sesión 5: chatbot + ToolNode
- Herramienta: buscar(consulta) en DuckDuckGo
- Un único prompt para todo el trabajo
- La tarea de A2A, con el formato

CONCEPTO CLAVE

Un agente único decide solo cuándo buscar, cuándo parar y cómo escribir. Todo en el mismo prompt.



FÍJATE EN

UNA sola búsqueda y a escribir. 2 llamadas al modelo, 284 s. Formato perfecto: 4/4 secciones y 4 citas.

thePower Tech School

Anatomía · un solo prompt para todo

01_agente_unico.py

```python
modelo = ChatOllama(model=MODELO, temperature=0, num_predict=700).bind_tools(tools)
# UN solo prompt para TODO: investigar, decidir cuando parar, ordenar y redactar con formato
SYSTEM = SystemMessage("Eres un asistente de investigacion. Responde en espanol. Usa la herramienta buscar "  (1)
                       "tantas veces como necesites y despues escribe el resumen final. " + FORMATO)
llamadas = 0


def chatbot(state: State) -> dict:
    global llamadas
    llamadas += 1
    return {"messages": [modelo.invoke([SYSTEM] + state["messages"])]}


builder = StateGraph(State)
builder.add_node("chatbot", chatbot)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "chatbot")
builder.add_conditional_edges("chatbot", tools_condition)  (2)
builder.add_edge("tools", "chatbot")
grafo = builder.compile()
```

1 **SYSTEM: investigar + redactar**

Busca, decide cuándo parar y escribe con formato: todo en un mensaje.

2 **El ciclo de la sesión 5**

tools_condition: ¿pide herramienta? -> tools -> vuelve al modelo.

thePower Tech School

Zoom · el informe del agente único

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\multiagent-team> python 01_agente_unico.py
=== INFORME
## Que es
El protocolo A2A (Agent2Agent) es un estándar abierto para la comunicación y colaboración
    entre agentes de inteligencia artificial (IA). Permite la interoperabilidad entre
    agentes de IA de diferentes proveedores o creados con diferentes marcos de agentes de IA
    [3]. Es un protocolo de comunicación abierto para sistemas multiagente que permite la
    colaboración efectiva entre agentes de IA [4].

## Quien lo impulsa
El protocolo A2A es impulsado por la comunidad de desarrolladores de inteligencia artificial
    y empresas como IBM, que buscan facilitar la comunicación y colaboración entre agentes
    de IA de diferentes orígenes [2].

## Como funciona
El protocolo A2A funciona mediante la definición de un lenguaje común para la comunicación
    entre agentes de IA. Esto permite que los agentes de IA de diferentes proveedores o
    creados con diferentes marcos de agentes de IA puedan comunicarse y colaborar de manera
    efectiva [1].

## A2A vs MCP
No se encontraron fuentes que compararan explícitamente el protocolo A2A con el MCP (Multi-
    Context Processing). Sin embargo, se puede inferir que el protocolo A2A se enfoca en la
    comunicación y colaboración entre agentes de IA, mientras que el MCP se enfoca en la
    integración de conocimientos y la resolución de problemas complejos en entornos
    multiagente. Es posible que el MCP sea un marco de trabajo más amplio que incluya la
    comunicación y colaboración entre agentes de IA como una de sus funcionalidades.

(.venv) PS C:\Users\ignaciode\multiagent-team>
```

SALIDA REAL

Formato perfecto... y dos errores

- Cumple los 4 títulos y cita [1]-[4]: la métrica da 4/4
- 'Quién lo impulsa': 'la comunidad y empresas como IBM'. Es Google
- 'A2A vs MCP': dice que no hay fuentes... y se inventa 'Multi-Context Processing'
- La métrica mide la FORMA, no si es verdad

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

El agente único saca 4/4 en formato. ¿Por qué decimos entonces que le cuesta?

Mirad cuántas búsquedas hizo y comparad lo que dice con el detector de mentiras.

thePower Tech School

Paso 1 · La respuesta

¿Por qué le cuesta a un agente solo?

- Buscó UNA vez: con eso no cubre cuatro preguntas distintas
- Nadie revisa: el mismo que escribe decide que está bien
- Rellena huecos inventando: 'Multi-Context Processing'
- La métrica de formato no lo detecta: hay que leer el contenido

thePower Tech School

Paso 1 · Lo que te llevas

- ✓ **El agente de la S5** mismo grafo, otra herramienta
- ✓ **Una búsqueda** 2 llamadas al modelo, 284 segundos
- ✓ **Formato 4/4** pero dos datos mal: quién lo impulsa y qué es MCP
- ✓ **Nadie revisa** el que escribe es el que decide

SIGUIENTE Repartimos el trabajo: un equipo con supervisor en LangGraph.

thePower Tech School

Paso 2 · Objetivo

2

OBJETIVO DEL PASO

Repartir el trabajo en un equipo con supervisor, con LangGraph

DE LA TEORÍA

Slides 5 y 7: el supervisor delega; el handoff y el estado común hacen que funcione.

LO QUE VAMOS A VER

- El estado compartido
- El supervisor y sus barandillas
- La primera versión (que falló)

ARCHIVO

- 02_equipo_langgraph.py

thePower Tech School

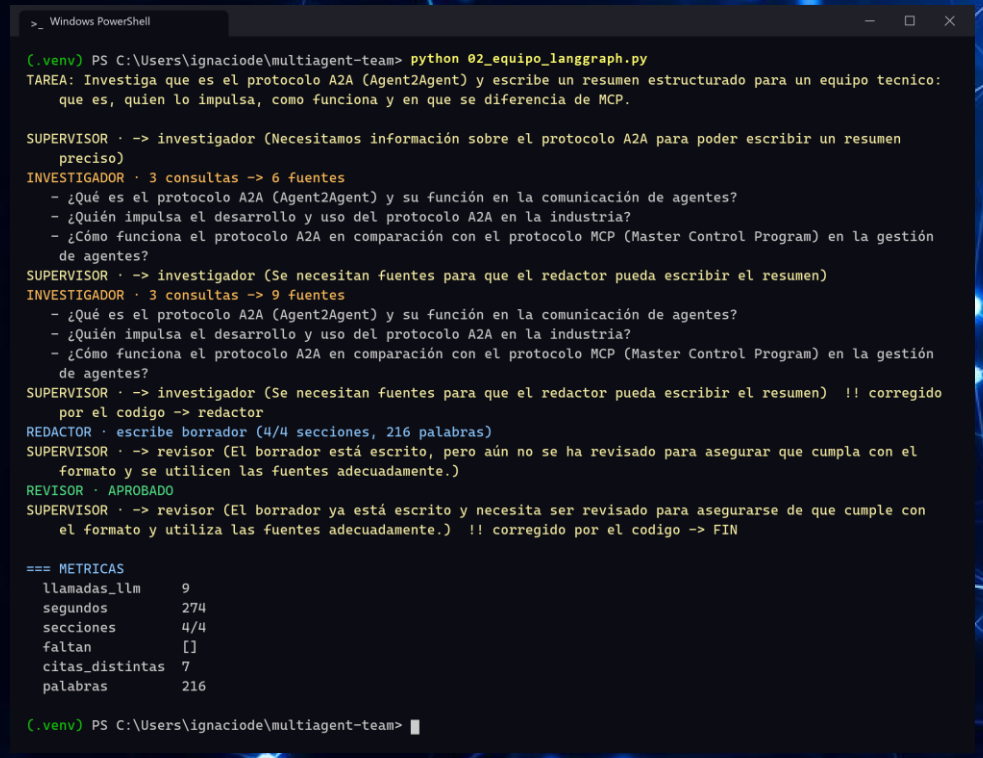
Paso 2 · Equipo de 3 agentes + supervisor con LangGraph

QUÉ HACEMOS

- Supervisor: decide quién trabaja (JSON)
- Investigador: 3 consultas y busca
- Redactor: escribe SOLO con las fuentes
- Revisor: aprueba o pide cambios

CONCEPTO CLAVE

Los trabajadores no hablan entre sí: leen y escriben en el estado común y devuelven el control al supervisor.

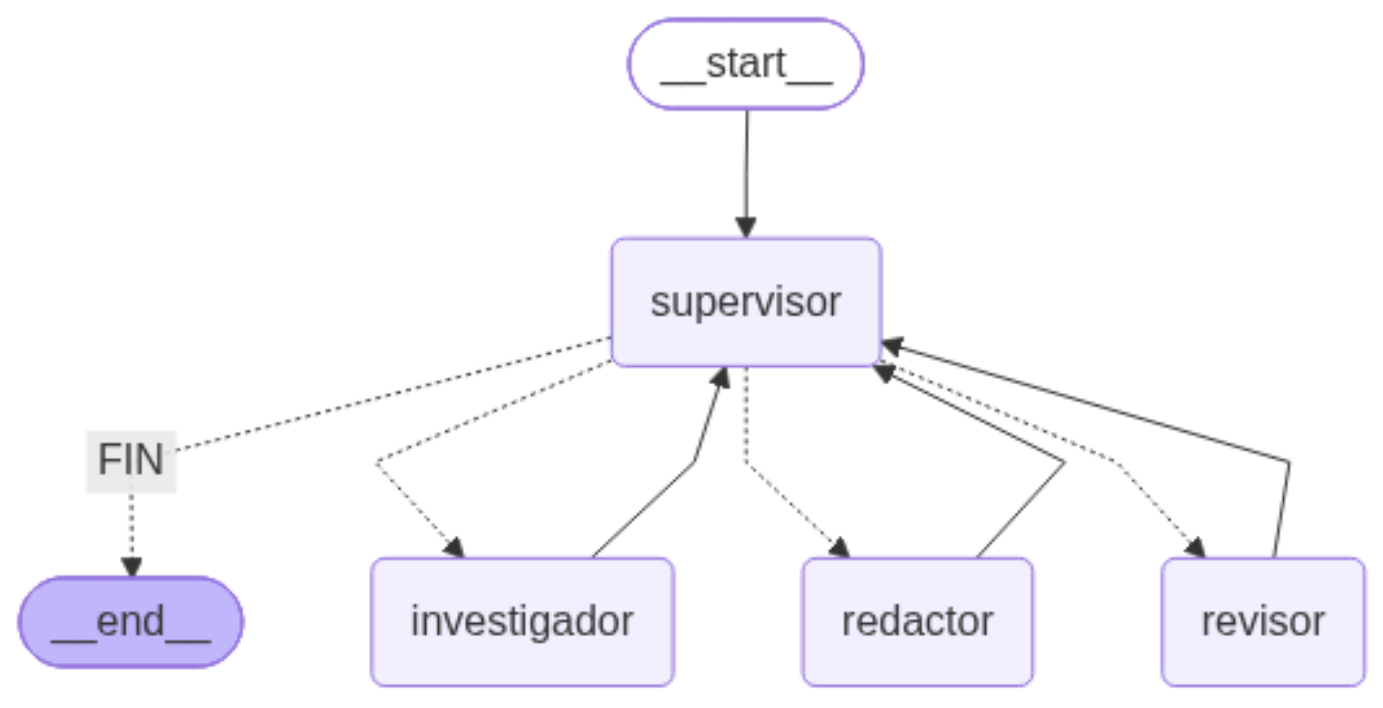


FÍJATE EN

Dos rondas de búsqueda, borrador, revisión APROBADA y fin. Dos veces el código corrige al supervisor (!!).

thePower Tech School

El grafo: una estrella alrededor del supervisor



CÓMO SE LEE

- Todo empieza y todo vuelve al supervisor.
- Flechas punteadas: la decisión del supervisor (arista condicional).
- Flechas continuas: el handoff de vuelta, siempre fijo.

Es el patrón supervisor de la slide 5, dibujado por LangGraph.

```
Lo genera el propio código: construir().get_graph().draw_mermaid_png()
```

thePower Tech School

Anatomía · la memoria compartida es el estado

02_equipo_langgraph.py

```python
class Estado(TypedDict, total=False):
    tarea: str
    fuentes: list          # lo que deja el investigador
    borrador: str          # lo que deja el redactor
    revision: dict         # lo que deja el revisor: {"aprobado": bool, "cambios": str}
    reescrituras: int
    rondas_inv: int
    siguiente: str         # la decision del supervisor
    turnos: int
    rastro: list           # quien hizo que, en orden (para verlo en clase)
    llamadas_llm: int
```

1

Lo que deja cada trabajador

fuentes (investigador), borrador (redactor), revisión (revisor).

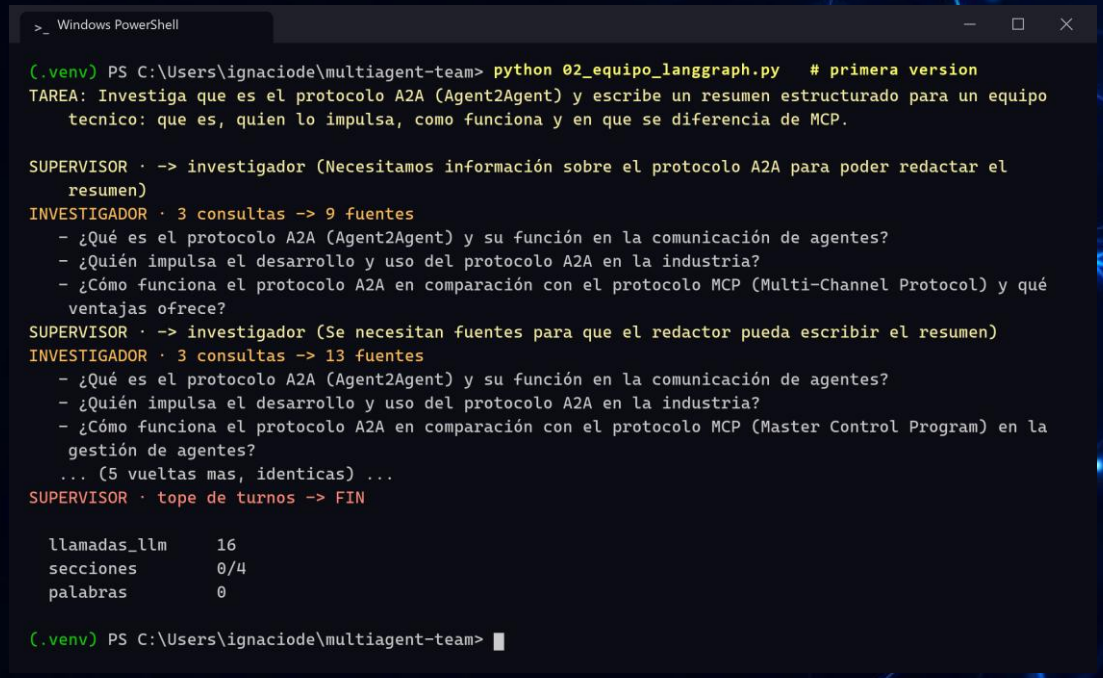
2

Lo que usa el supervisor

siguiente, turnos y el rastro de todo lo que ha pasado.

thePower Tech School

La primera versión: el supervisor en bucle



QUÉ PASÓ (EJECUCIÓN REAL)

8 veces 'investigador', con 17 fuentes ya encontradas. 16 llamadas, 296 s y un informe vacío. Solo lo paró el tope de turnos.

POR QUÉ

Le dejábamos pedir más investigación sin límite. Un modelo de 8B repite la decisión que le parece segura: 'faltan fuentes'.

thePower Tech School

Anatomía · el supervisor y sus barandillas

02_equipo_langgraph.py

```python
def regla(e: Estado) -> str:
    """Lo que haria un jefe sensato. Lo usamos para CORREGIR al supervisor si elige algo imposible."""
    if not e.get("fuentes"):                                                                    # 1
        return "investigador"
    if not e.get("borrador"):
        return "redactor"
    if not e.get("revision"):
        return "revisor"
    if not e["revision"]["aprobado"] and e.get("reescrituras", 0) < MAX_REESCRITURAS:
        return "redactor"
    return "FIN"


def supervisor(e: Estado) -> Estado:
    turnos = e.get("turnos", 0) + 1
    if turnos > MAX_TURNOS:
        return {"siguiente": "FIN", "turnos": turnos, "rastro": e["rastro"] + ["SUPERVISOR · tope de turnos -> FIN"]}
    r = json.loads(juez.invoke(SUPERVISOR.format(tarea=e["tarea"], **resumen_estado(e))).content)   # 2
    eleccion, motivo = r.get("siguiente", ""), r.get("motivo", "")
    linea = f"SUPERVISOR · -> {eleccion} ({motivo})"
    sensata = regla(e)
    # Barandillas en codigo: no se redacta sin fuentes, no se acaba sin revisar, no se reescribe sin fin.
    # Unica libertad: pedir UNA ronda mas de investigacion antes del borrador. En la primera version le dejabamos
    # investigar sin limite y el supervisor mando 8 veces al investigador (17 fuentes, 0 informe)
    extra_ok = eleccion == "investigador" and not e.get("borrador") and e.get("rondas_inv", 0) < MAX_RONDAS_INV   # 3
    if eleccion != sensata and not extra_ok:
        linea += f"  !! corregido por el codigo -> {sensata}"
        eleccion = sensata
    return {"siguiente": eleccion, "turnos": turnos, "llamadas_llm": e["llamadas_llm"] + 1,
            "rastro": e["rastro"] + [linea]}
```

1 regla(): lo que haría un jefe sensato

Sin fuentes, investigar; sin borrador, redactar; sin revisión, revisar.

2 El LLM propone

Decide en JSON quién sigue y por qué.

3 El código dispone

Si elige algo imposible, se corrige (!!). Solo una ronda extra de búsqueda.

thePower Tech School

Anatomía · el handoff son aristas

02_equipo_langgraph.py

```python
def construir():
    g = StateGraph(Estado)
    g.add_node("supervisor", supervisor)
    for nombre, fn in (("investigador", investigador), ("redactor", redactor), ("revisor", revisor)):
        g.add_node(nombre, fn)
        g.add_edge(nombre, "supervisor")        # handoff de vuelta: todos devuelven el control al jefe  (1)
    g.add_edge(START, "supervisor")
    g.add_conditional_edges("supervisor", lambda e: e["siguiente"],                                      (2)
                            {"investigador": "investigador", "redactor": "redactor", "revisor": "revisor", "FIN": END})
    return g.compile()
```

1 **add_edge(trabajador, supervisor)**
Cada trabajador devuelve SIEMPRE el control al jefe.

2 **add_conditional_edges**
El jefe elige a quién pasa el trabajo, o END.

thePower Tech School

Zoom · el informe del equipo

Windows PowerShell

```
(.venv) PS C:\Users\ignaciode\multiagent-team> python 02_equipo_langgraph.py
=== INFORME
## Que es
El protocolo A2A (Agent2Agent) es un protocolo de comunicación abierto para agentes de
    inteligencia artificial (IA) diseñado para sistemas multiagente, que permite la
    interoperabilidad entre agentes de IA de diversos proveedores o aquellos creados
    utilizando diferentes marcos de agentes de IA [1]. El protocolo A2A permite a los
    agentes de IA comunicarse, colaborar y delegarse tareas entre sí sin intervención humana
    [5].

## Quien lo impulsa
El protocolo A2A fue presentado inicialmente por Google en abril de 2025 [3] y está
    impulsado por Google [5]. Actualmente, el protocolo A2A está contribuido a la Fundación
    Linux [6].

## Como funciona
El protocolo A2A funciona mediante la publicación de un documento en JSON llamado Agent
    Card, que declara las habilidades del agente, la ruta de comunicación y el mecanismo de
    autenticación [2]. Los agentes pueden entonces comunicarse y delegarse tareas entre sí
    de manera segura e interoperable [4].

## A2A vs MCP
El protocolo A2A se diferencia del protocolo MCP (Multi-Cloud Platform) en que A2A se enfoca
    en la comunicación y colaboración entre agentes de IA, mientras que MCP se enfoca en la
    conexión de agentes con herramientas y datos [9]. En un sistema real, se pueden apilar
    ambos protocolos, ya que cada agente puede poseer sus herramientas vía MCP y cooperar
    vía A2A [9].

(.venv) PS C:\Users\ignaciode\multiagent-team>
```

SALIDA REAL

Ahora sí: Google, Linux Foundation y Agent Card

- Quién lo impulsa: Google en abril de 2025, hoy en la Linux Foundation
- Cómo funciona: la Agent Card, un JSON con lo que sabe hacer el agente
- A2A vs MCP: MCP conecta con herramientas y datos, A2A entre agentes. Correcto
- Pero sigue expandiendo mal las siglas: 'Multi-Cloud Platform'

thePower Tech School

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

Si el código acaba corrigiendo al supervisor, ¿para qué queremos un supervisor que sea un LLM?

Pensad en tareas en las que el orden no se puede saber de antemano.

thePower Tech School

Paso 2 · La respuesta

¿LLM de supervisor o reglas?

- Hoy el orden era casi fijo: unas reglas bastaban
- El LLM compensa cuando hay que decidir con el contenido: '¿falta algo?'
- Con un modelo pequeño: el LLM propone y el código pone límites
- Topes siempre: de turnos, de rondas, de reescrituras

thePower Tech School

Paso 2 · Lo que te llevas

- ✓ **Patrón supervisor** un jefe y tres trabajadores que no hablan entre sí
- ✓ **Estado común** fuentes, borrador y revisión en un TypedDict
- ✓ **Barandillas** la primera versión entró en bucle: 16 llamadas, 0 informe
- ✓ **Mejor informe** 9 llamadas, 274 s, 7 citas y los datos correctos

SIGUIENTE El mismo equipo, pero sin escribir el grafo: CrewAI.

thePower Tech School

Paso 3 · Objetivo

3

OBJETIVO DEL PASO

Montar el mismo equipo con CrewAI y ver qué nos ahorra (y qué no)

DE LA TEORÍA

Slides 8 y 9: CrewAI define equipos por roles; muy rápido de montar, menos control fino.

LO QUE VAMOS A VER

- Agentes: rol, objetivo, historia
- Tareas y su contexto
- Secuencial frente a jerárquico

ARCHIVO

- 03_equipo_crewai.py

thePower Tech School

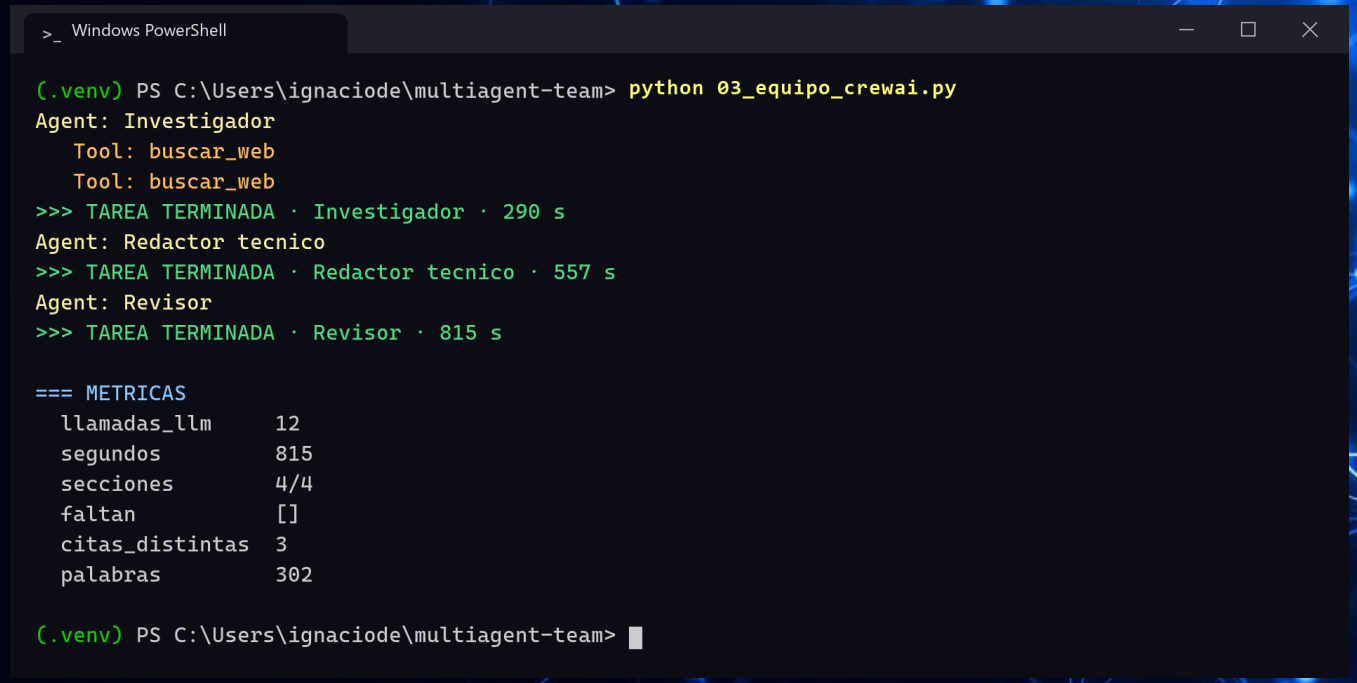
Paso 3 · El mismo equipo con CrewAI

QUÉ HACEMOS

- pip install crewai (sin pagar nada)
- 3 agentes: rol + objetivo + historia
- 3 tareas encadenadas con context=[...]
- El mismo llama3.1 local vía LiteLLM

CONCEPTO CLAVE

No escribimos estado, nodos ni aristas: describimos quién es cada uno y qué tiene que entregar.



FÍJATE EN

2 búsquedas, 12 llamadas y 815 s: el triple que LangGraph. Formato 4/4 pero solo 3 citas.

thePower Tech School

Anatomía · un agente de CrewAI

03_equipo_crewai.py

```python
investigador = Agent(
    role="Investigador",
    goal="Encontrar en la web datos concretos y fiables sobre la tarea",
    backstory="Analista tecnico que siempre anota de donde saca cada dato.",
    tools=[buscar], llm=llm, max_iter=4, allow_delegation=False, verbose=True)
```

1

2

1 role, goal, backstory

El 'organigrama': quién es y qué busca. Va dentro de su prompt.

2 tools y max_iter

Sus herramientas y cuántas vueltas puede dar como mucho.

thePower Tech School

Anatomía · las tareas y el handoff

03_equipo_crewai.py

```python
investigar = Task(
    description="Busca informacion para esta tarea: {tarea} Haz entre 2 y 3 busquedas distintas.",
    expected_output="Como mucho 6 fuentes numeradas [1], [2]..., cada una en UNA linea: titulo, url y su dato clave.",  # 1
    agent=investigador)
redactar = Task(
    description="Con las notas del investigador, escribe el resumen de la tarea: {tarea} NO copies las notas: "
                "redacta el resumen. " + FORMATO,
    expected_output="El resumen en markdown con los 4 titulos y citas [n].",
    agent=redactor, context=[investigar])  # 2
revisar = Task(
    description="Revisa el resumen: que tenga los 4 titulos y que cada dato lleve cita [n]. "
                "Corrige lo que falle. " + FORMATO,
    expected_output="La version final del resumen, en markdown, y nada mas.",
    agent=revisor, context=[investigar, redactar])
```

1 **expected_output**

Qué tiene que entregar: es su 'contrato'.

2 **context=[...]**

El handoff de CrewAI: la salida de una tarea entra en la siguiente.

thePower Tech School

Zoom · el informe de CrewAI

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\multiagent-team> python 03_equipo_crewai.py
=== INFORME
## Que es
El protocolo A2A (Agent2Agent) es un estándar abierto que permite a los agentes de IA
    delegar tareas entre frameworks [1]. Esto significa que los agentes pueden comunicarse y
    colaborar entre sí de manera segura y eficiente [1]. El protocolo A2A es una forma de
    que los agentes de IA puedan trabajar juntos para lograr objetivos comunes [1].

## Quien lo impulsa
A2A (Agent2Agent) es impulsado por la comunidad de desarrolladores de IA y tecnología de
    inteligencia artificial [2]. El protocolo A2A es un estándar abierto, lo que significa
    que cualquier desarrollador puede contribuir y mejorar el protocolo [2]. La comunidad de
    A2A trabaja para hacer que el protocolo sea más seguro, escalable y fácil de usar [2].

## Como funciona
El protocolo A2A funciona delegando tareas entre frameworks [1]. Los agentes de IA se
    comunican entre sí y pueden delegar tareas a otros agentes que tengan la capacidad de
    realizarlas [1]. Esto permite que los agentes de IA trabajen juntos para lograr
    objetivos comunes de manera eficiente y segura [1]. El protocolo A2A también permite que
    los agentes de IA se descubran y colaboren entre sí [3].

## A2A vs MCP
A2A (Agent2Agent) se diferencia de MCP (Multi-Cloud Platform) en que A2A es un protocolo de
    comunicación entre agentes de IA, mientras que MCP es una plataforma de nube que permite
    la integración de diferentes servicios y aplicaciones [2]. A2A es un estándar abierto,
    mientras que MCP es una plataforma propietaria [2]. A2A permite la delegación de tareas
    entre frameworks, mientras que MCP se enfoca en la integración de servicios y
    aplicaciones [2].

**Referencias:**

[1] Protocolo Agent2Agent: Seguridad A2A, arquitectura y comparación con MCP
                          (https://www.openlegion.ai/es/learn/agent2agent-protocol)
[2] ¿Qué es A2A? El protocolo entre agentes frente a MCP
    (https://arte.itlibra.com/es/articles/what-is-a2a-protocol)
[3] Qué es el protocolo A2A (Agent2Agent) y cuándo lo necesitas
    (https://blog.shakersworks.com/que-es-el-protocolo-a2a)

(.venv) PS C:\Users\ignaciode\multiagent-team>
```

SALIDA REAL

Formato perfecto, contenido flojo

- 4/4 secciones, pero solo 3 fuentes citadas (LangGraph: 7)
- 'Quién lo impulsa': 'la comunidad'. Ni rastro de Google
- MCP: 'Multi-Cloud Platform, una plataforma propietaria'. Falso
- Sin barandillas propias: nadie comprobó el contenido

thePower Tech School

Por qué CrewAI tardó tanto en mi portátil

1. **Prompts mucho más largos**
   Cada agente lleva su rol, su historia, el formato ReAct y la salida de las tareas anteriores. Más texto = más segundos en CPU.

2. **Más vueltas por agente**
   El investigador razona, llama a la herramienta y vuelve a razonar: cada vuelta es una llamada al modelo.

3. **La primera ejecución no terminó**
   A los 20 minutos la cortamos, y el redactor había copiado las notas en vez de escribir. Lo arreglamos acortando las notas y siendo explícitos.

4. **Con un modelo de API sería otra historia**
   CrewAI está pensado para modelos grandes y rápidos. Con un 8B en CPU se nota su coste.

thePower Tech School

Anatomía · secuencial o jerárquico (supervisor automático)

03_equipo_crewai.py

```python
    if jerarquico:
        # Supervisor automatico: un agente "manager" reparte las tareas usando la herramienta de delegar
        for t in (investigar, redactar, revisar):
            t.agent = None
        crew = Crew(agents=[investigador, redactor, revisor], tasks=[investigar, redactar, revisar],
                    process=Process.hierarchical, manager_llm=llm, verbose=True)  # 1
    else:
        crew = Crew(agents=[investigador, redactor, revisor], tasks=[investigar, redactar, revisar],
                    process=Process.sequential, verbose=True)  # 2

    t0 = time.time()
    # Al acabar cada tarea, cuanto llevamos: asi vemos donde se va el tiempo
    crew.task_callback = lambda salida: print(f"\n>>> TAREA TERMINADA · {salida.agent} · {round(time.time() - t0)} s\n")
    resultado = crew.kickoff(inputs={"tarea": TAREA})
```

1 **Process.hierarchical + manager_llm**

CrewAI crea un 'manager' que reparte las tareas: el supervisor, sin escribirlo.

2 **Process.sequential**

Cadena de montaje: una tarea detrás de otra. Es el que ejecutamos.

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

CrewAI se escribe más rápido. ¿Por qué no lo usamos siempre?

Recordad lo que tuvimos que añadir al supervisor de LangGraph para que no entrara en bucle.

thePower Tech School

Paso 3 · La respuesta

¿CrewAI siempre?

- El flujo lo decide el framework: menos sitio para barandillas propias
- Solo tienes mandos generales: max_iter, context, tipo de proceso
- Prompts más largos: más coste y más lentitud
- Para prototipar un equipo por roles, imbatible

thePower
Tech School

Paso 3 · Lo que te llevas

- ✓ **Roles y tareas** sin estado, sin nodos, sin aristas
- ✓ **context** el handoff de CrewAI
- ✓ **Secuencial o jerárquico** el jerárquico crea el supervisor solo
- ✓ **El precio** 12 llamadas y 815 s: el triple que LangGraph

SIGUIENTE Ponemos los tres uno al lado del otro, con números.

thePower Tech School

Paso 4 · Objetivo

4

OBJETIVO DEL PASO

Comparar las implementaciones con números, no con opiniones

DE LA TEORÍA

Slide 9: control frente a rapidez. Slide 14: coordinar cuesta.

LO QUE VAMOS A VER

- Líneas de código
- Llamadas, segundos y calidad
- Cuándo elegir cada uno

ARCHIVO

- 04_comparar.py

thePower Tech School

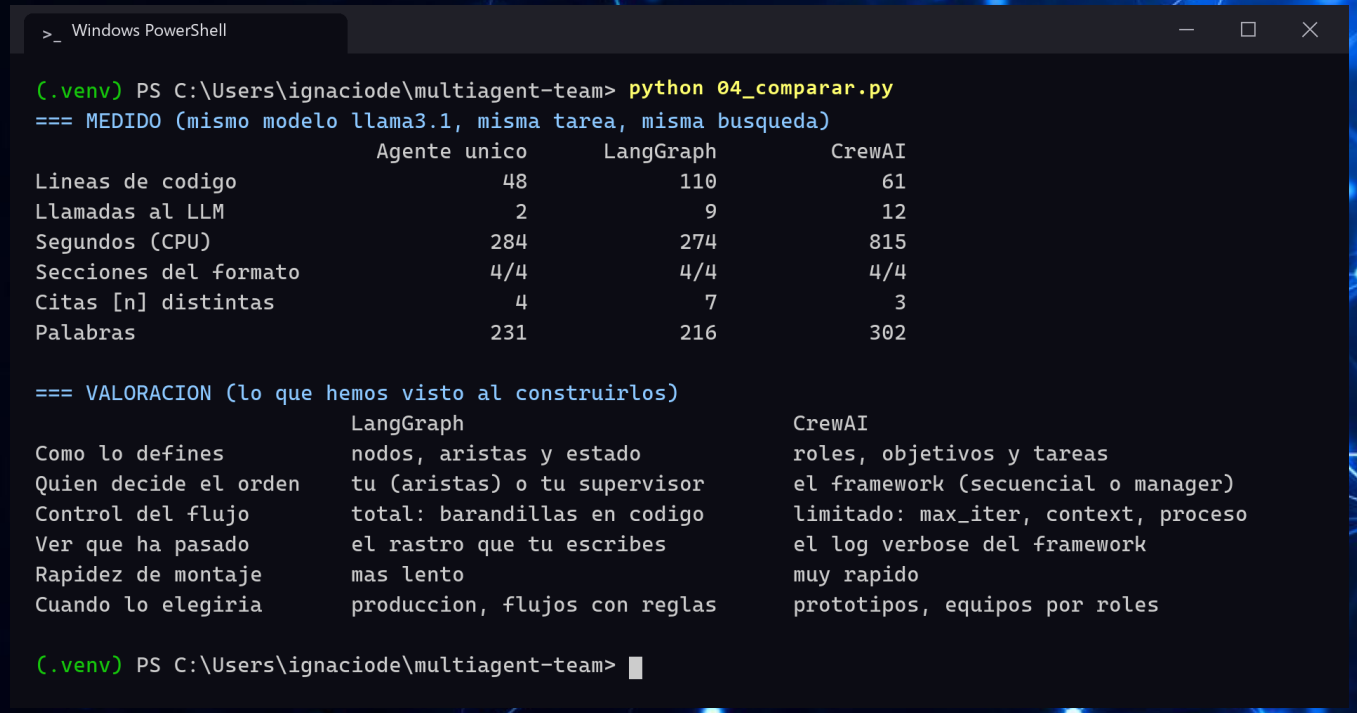
Paso 4 · Comparar LangGraph y CrewAI

QUÉ HACEMOS

- Lee las métricas guardadas en salidas/
- Cuenta las líneas de código reales
- No llama al modelo: es instantáneo
- Tabla medida + tabla de valoración

CONCEPTO CLAVE

Mismo modelo, misma búsqueda, misma medida: lo único que cambia es cómo se coordina el equipo.



FÍJATE EN

LangGraph: más código (110 líneas) y el mejor informe. CrewAI: 61 líneas y el triple de tiempo.

thePower Tech School

Cuándo elegiría cada uno

	LangGraph	CrewAI
Lo defines con	nodos, aristas y estado	roles, objetivos y tareas
Quién decide el orden	tú, o tu supervisor con tus reglas	el framework
Control del flujo	total: barandillas en código	mandos generales
Depurar	el rastro que tú escribes	el log verbose
Montarlo	más lento	muy rápido
Lo elegiría para	producción, reglas de negocio	prototipos, equipos por roles

Y AutoGen (slide 8): agentes que conversan entre sí. Mismo problema, tercera filosofía.

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

El equipo hace 4 veces más llamadas que el agente único. ¿Cuándo compensa?

Pensad en lo que costaría el error de 'Multi-Context Processing' en un informe para un cliente.

thePower Tech School

Paso 4 · La respuesta

¿Cuándo compensa un equipo?

- Cuando la tarea tiene fases de verdad: buscar, escribir, revisar
- Cuando un error cuesta más que unas llamadas extra
- Si la tarea es simple, un agente solo: más barato y más rápido
- Medir siempre: llamadas, tiempo y calidad (slide 14)

thePower
Tech School

Paso 4 · Lo que te llevas

- ✓ **Misma tarea, misma medida** si no, no se puede comparar
- ✓ **LangGraph** más código, más control, el más rápido hoy
- ✓ **CrewAI** menos código, más lento con un 8B en CPU
- ✓ **Un equipo cuesta** más llamadas: solo si la tarea lo merece

SIGUIENTE Cerramos: patrones, handoff y el material.

thePower Tech School

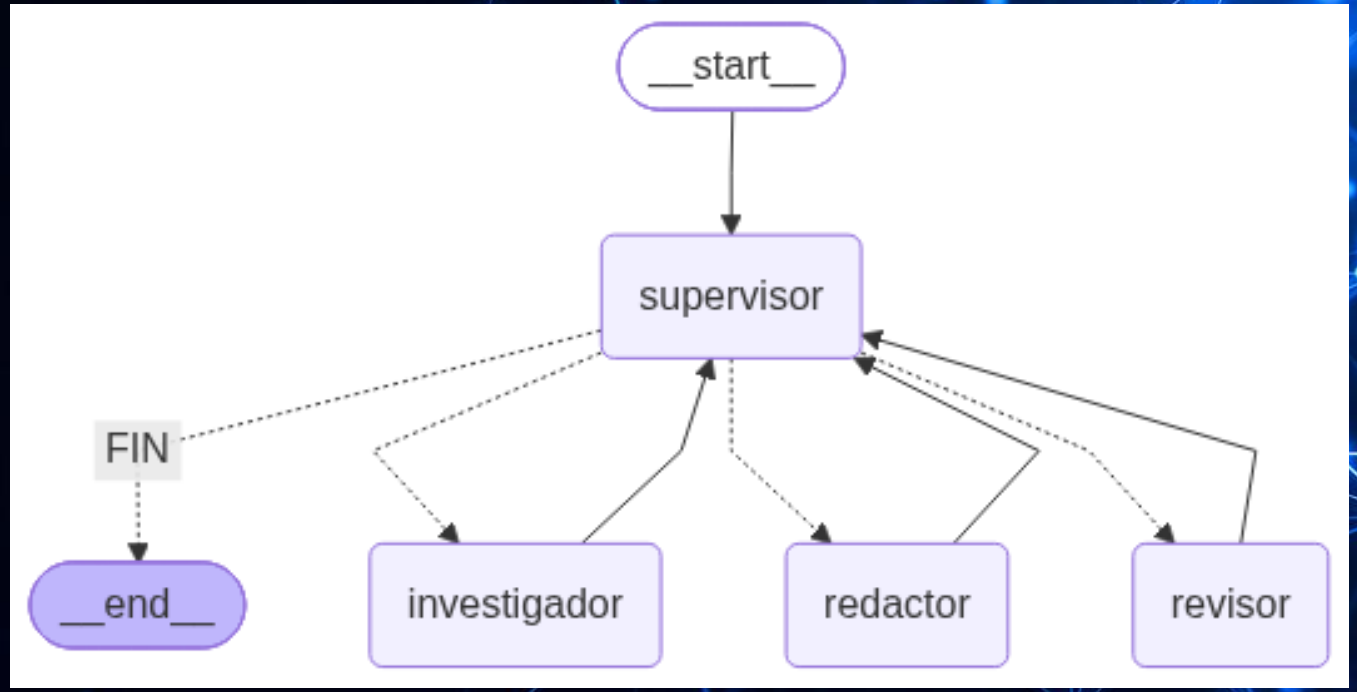
Paso 5 · Cierre y material

QUÉ HACEMOS

- Patrones: supervisor, jerárquico, secuencial
- Handoff y memoria compartida en el código
- README con la comparativa
- Los dos ejercicios

CONCEPTO CLAVE

Un equipo de agentes es un grafo con reglas. El framework decide cuánto de ese grafo escribes tú.



FÍJATE EN

El mismo supervisor de la slide 5 de teoría, ahora con código que funciona y barandillas.

thePower Tech School

Recap: de la teoría al código

Teoría	En el código	Lo que vimos de verdad
Slide 4 · Agente único	01_agente_unico.py	1 búsqueda, se inventó qué es MCP
Slide 5 · Supervisor	supervisor() + regla()	el LLM propone, el código dispone
Slide 6 · Secuencial	Process.sequential / ejercicio 1	cadena de montaje con context
Slide 7 · Handoff y estado	Estado + aristas de vuelta	fuentes, borrador y revisión
Slide 9 · LangGraph vs CrewAI	02 frente a 03	110 líneas y 274 s frente a 61 líneas y 815 s
Slide 11 · Coordinar cuesta	tope de turnos	sin él: 16 llamadas y 0 informe

thePower Tech School

Paso 5 · Lo que te llevas

- ✓ **Del agente al equipo** cuando la tarea tiene fases
- ✓ **Supervisor + estado común** el patrón más usado
- ✓ **Barandillas** topes y reglas: un equipo sin ellas no acaba
- ✓ **Framework según el caso** control con LangGraph, rapidez con CrewAI

SIGUIENTE Deberes: un equipo de dos y, el reto, el soporte de pedidos en los dos frameworks.

thePower Tech School

Ejercicio 1 · Un equipo de 2 agentes

QUÉ HACER

MEDIO · 2 h

1. **Define los 2 roles**
   en la solución: investigador y redactor

2. **Conéctalos**
   estado compartido; en secuencia o con supervisor

3. **Lanza una tarea real**
   la solución: la semana laboral de 4 días

4. **Observa el handoff**
   qué le pasa uno al otro

ENTREGABLE

Repositorio + la salida final del equipo + 2 frases: qué framework usaste y por qué.

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\multiagent-team> python soluciones/ejercicio1_dos_agentes.py
INVESTIGADOR · 6 fuentes
REDACTOR · resumen escrito

=== RESUMEN
Aquí te presento un resumen de 5 líneas sobre la semana laboral de 4 días en España, basado en
    las fuentes proporcionadas:

La semana laboral de 4 días es una forma de organizar la jornada de trabajo semanal,
    estableciendo 4 días laborables [1]. Varias pymes industriales en España han probado esta
    semana laboral con ayudas públicas y han obtenido resultados positivos, como una reducción
    de bajas y una mayor retención de plantilla [2]. La semana laboral de 4 días está ganando
    popularidad en España y se está implementando en varios sectores [3]. Los pilotos de esta
    semana laboral han demostrado que es posible mejorar la productividad y el bienestar de los
    trabajadores [5]. Sin embargo, no todas las empresas están dispuestas a adoptar esta nueva
    forma de trabajo, y algunos sectores pueden seguir utilizando la jornada laboral de 5 días
    [6].

(89 s, 1 llamada al LLM)

(.venv) PS C:\Users\ignaciode\multiagent-team>
```

ASÍ DEBE VERSE (SOLUCIÓN REAL)

Investigador: 6 fuentes. Redactor: 5 líneas con citas [1]-[6]. 89 s y una sola llamada al modelo.

thePower Tech School

Ejercicio 2 · Reto: soporte de pedidos en equipo

QUÉ CONSTRUIR

RETO

1. **Parte del agente de la S5**
   el de consulta_pedido

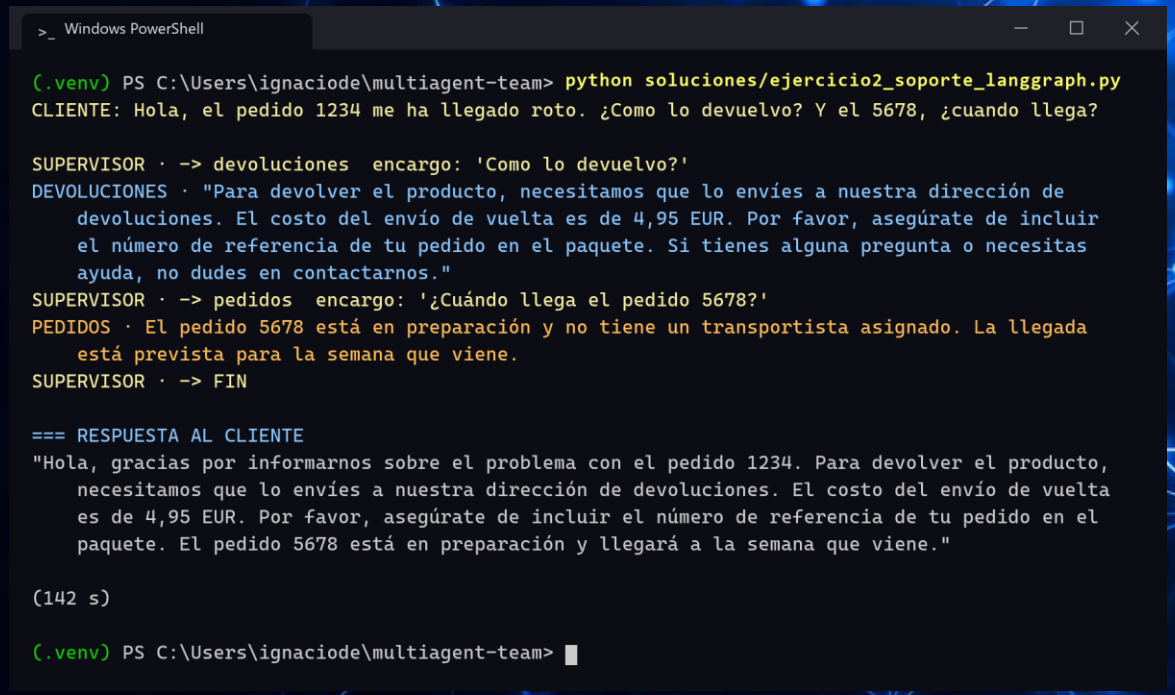
2. **Un equipo de 2-3 agentes**
   pedidos + devoluciones + quién responde

3. **En LangGraph Y en CrewAI**
   el mismo equipo, el mismo mensaje

4. **Tabla comparativa**
   control, rapidez de desarrollo y coste

ENTREGABLE

Repo con las dos versiones del mismo equipo + tabla (control, rapidez, coste). Solución en soluciones/.



LA SOLUCIÓN LANGGRAPH, EJECUTADA

El supervisor reparte un encargo a cada uno. Pero el encargo pierde 'roto': cobra 4,95 EUR en vez de gratis.

thePower Tech School

Test de repaso · 1/10

TEST · ELIGE UNA RESPUESTA

Según la teoría de la sesión, ¿en qué situación se satura un agente único y empieza a compensar un equipo?

- **A** Siempre que la tarea use al menos una herramienta externa
- **B** Cuando la tarea tiene varias fases distintas, como buscar, redactar y revisar
- **C** Cuando el modelo tiene menos de 70.000 millones de parámetros
- **D** Cuando la respuesta final supera las mil palabras

thePower Tech School

Test de repaso · 2/10

TEST · ELIGE UNA RESPUESTA

¿Qué patrón de coordinación elegiríais si el flujo es siempre el mismo y nunca hay que volver atrás?

- **A** Supervisor, porque es el patrón más común y flexible
- **B** Jerárquico, con un supervisor de supervisores
- **C** Conversacional, al estilo AutoGen, con debate entre agentes
- **D** Secuencial, como una cadena de montaje

thePower Tech School

Test de repaso · 3/10

TEST · ELIGE UNA RESPUESTA

En el equipo de LangGraph de la práctica, ¿cómo se pasan el trabajo los agentes trabajadores entre sí?

A. No hablan entre sí: escriben en el estado común y devuelven el control al supervisor

B. El investigador llama directamente al redactor cuando termina su búsqueda

C. Cada trabajador recibe la conversación completa de todos los demás

D. El revisor envía el borrador corregido directamente al cliente

thePower Tech School

Test de repaso · 4/10

TEST · ELIGE UNA RESPUESTA

¿Qué ocurrió en la primera versión del supervisor de LangGraph, la que no tenía barandillas?

- **A** Mandó directamente al redactor sin buscar ninguna fuente
- **B** Terminó bien, pero tardó más de veinte minutos en CPU
- **C** Mandó ocho veces al investigador y acabó con 16 llamadas y un informe vacío
- **D** El revisor rechazó el borrador una y otra vez sin parar

thePower Tech School

Test de repaso · 5/10

TEST · ELIGE UNA RESPUESTA

¿Cómo se arregló el bucle del supervisor en el equipo de LangGraph?

A Con una regla en código que corrige al modelo y topes de rondas y turnos

B Cambiando llama3.1 por un modelo de pago mucho más grande

C Eliminando al revisor para que el flujo fuera más corto

D Subiendo la temperatura del supervisor para que variara sus decisiones

thePower Tech School

Test de repaso · 6/10

TEST · ELIGE UNA RESPUESTA

Los tres informes sacaron 4/4 en la métrica de formato. ¿Qué nos enseñó eso sobre medir?

A. Que los tres informes eran igual de buenos y fiables

B. Que la métrica estaba mal programada y había que cambiarla

C. Que con cuatro citas o más un informe ya es fiable

D. Que la métrica mide la forma, no la verdad: además de medir hay que leer

thePower Tech School

Test de repaso · 7/10

TEST · ELIGE UNA RESPUESTA

Con el mismo llama3.1 en CPU, ¿por qué CrewAI tardó 815 s frente a los 274 s de LangGraph?

A. Porque CrewAI envía las consultas a un servidor en la nube

B. Porque sus prompts son mucho más largos y cada agente da más vueltas

C. Porque CrewAI ejecuta el proceso jerárquico por defecto

D. Porque DuckDuckGo limita las búsquedas que llegan desde CrewAI

thePower Tech School

Test de repaso · 8/10

TEST · ELIGE UNA RESPUESTA

En CrewAI, ¿qué pieza hace de handoff entre una tarea y la siguiente?

A El backstory de cada agente, que describe a quién pasa el trabajo

B El parámetro max_iter, que fija cuántas vueltas puede dar cada agente

C El context de cada tarea, que le pasa la salida de las tareas anteriores

D Una arista condicional que se dibuja entre los agentes

Test de repaso · 9/10

TEST · ELIGE UNA RESPUESTA

En el reto de soporte de pedidos, ¿por qué el cliente con el pedido roto acabó con un cobro de 4,95 EUR?

- **A** Porque la herramienta de devoluciones tenía mal programada la política
- **B** Porque el agente de pedidos respondió también a la devolución
- **C** Porque el supervisor terminó antes de llamar al agente de devoluciones
- **D** Porque el encargo que pasó el supervisor perdió la palabra «roto»

thePower Tech School

Test de repaso · 10/10

TEST · ELIGE UNA RESPUESTA

Para un sistema en producción con reglas de negocio obligatorias, ¿qué framework recomendamos y por qué?

A LangGraph, porque podéis meter vuestras reglas en código en cualquier punto del flujo

B CrewAI, porque con menos líneas de código hay menos sitios donde fallar

C AutoGen, porque la conversación entre agentes evita los bucles

D Cualquiera de ellos, porque todos ofrecen exactamente el mismo control

thePower Tech School